# Informe de laboratorio — Analizador léxico

**Integrante:** Paulo Araya
**Fecha:** 11 de noviembre de 2026
**IDE y Java:** Visual Studio Code, JDK 27 de Temurin, compilación `--release 17`.

## 1. Diagnóstico inicial

La implementación inicial concentraba el estado de la tokenización (`source`, `cursor` y `tokens`) en atributos de instancia. `tokenize` decidía todos los símbolos y delegaba palabras y números en métodos que también dependían de ese estado. Además, incluía responsabilidades de depuración y conteo de caracteres. Las pruebas diagnósticas mostraron cuatro casos correctos de diez. Los defectos principales fueron la clasificación por prefijo `let`, la falta de soporte para guion bajo, el consumo de un solo dígito, el descarte silencioso de caracteres inválidos y la acumulación de tokens entre llamadas.

## 2. Pruebas automatizadas

| Test | Entrada | Esperado | Inicial | Final |
|---|---|---|---|---|
| 01 | `""` | lista vacía | pasa | pasa |
| 02 | `let print` | dos `KEYWORD` | pasa | pasa |
| 03 | `letter` | `IDENTIFIER` completo | falla | pasa |
| 04 | `_total2` | un `IDENTIFIER` | falla | pasa |
| 05 | `2026` | un `INTEGER` | falla | pasa |
| 06 | `=+();` | cinco símbolos | pasa | pasa |
| 07 | expresión completa | siete tokens | falla | pasa |
| 08 | espacios y posiciones | cuatro tokens | pasa | pasa |
| 09 | `abc@5` | `ERROR` en `@` | falla | pasa |
| 10 | llamadas `x`, luego `y` | solo `y` | falla | pasa |

Se agregaron cinco pruebas de frontera: mayúsculas, límites de palabras, guion bajo y dígitos, errores consecutivos y caracteres no ASCII. Todas pasan.

### Prueba con Mockito

`KeywordCatalog` se reemplazó por un mock. `custom` devuelve `true` y `other` devuelve `false` mediante `when(...).thenReturn(...)`; después se verifica con `verify(...)` que ambas palabras fueron consultadas. Esto aísla la dependencia del conjunto concreto de palabras reservadas y prueba que el analizador utiliza el catálogo.

## 3. Correcciones y refactorizaciones

| Problema | Cambio | Evidencia |
|---|---|---|
| Palabras reservadas por prefijo | Usar `catalog.isKeyword` sobre la palabra completa | 03, 11, 12 |
| Reglas Unicode implícitas y `_` inválido | Predicados ASCII explícitos | 04, 13, 15 |
| Números de un carácter | Consumir todos los dígitos contiguos | 05, 07 |
| Caracteres inválidos descartados | Emitir un `ERROR` por carácter | 09, 14, 15 |
| Estado compartido | Usar variables locales en `tokenize` | 10 |
| Responsabilidades mezcladas | Extraer mapeo de símbolos/predicados y eliminar depuración no usada | métricas y suite completa |

## 4. Métricas

Se usó `scripts/metrics.py` con el mismo criterio en ambos estados: LOC no vacías ni exclusivamente de comentario; LCOM1 por acceso directo a atributos; CBO incluyendo `String` y biblioteca estándar; y CC con `if`, `while`, `case`, `&&`, `||` y ternario.

| Indicador | Inicial | Final | Interpretación |
|---|---|---|---|
| LOC de `LexicalAnalyzer` | 68 | 70 | aumentan por reglas explícitas y separación |
| LOC total del paquete | 129 | 131 | mayor explicitud, no peor calidad automática-mente |
| LCOM1 | 6 | 1 | se elimina estado y código ajeno |
| CBO | 7 | 6 | desaparece la dependencia de `Character` |

| Indicador | Inicial | Final | Interpretación |
|---|---|---|---|
| CC máxima | 11 (`tokenize`) | 6 (`symbolType`) | baja la concentración de decisiones |
| Suma de CC | 21 | 32 | predicados explícitos agregan decisiones |
| Tests aprobados / ejecutados | 4/10 diagnóstico | 17/17 | contrato cubierto y verde |

La tabla LCOM1 final contiene `tokenize -> vacio` y `readWord -> catalog`; su único par es disjunto, por lo que LCOM1 = 1. El máximo de CC corresponde al `switch` plano de `symbolType`, cuyo valor sobreestima la complejidad cognitiva real: cada `case` representa un mapeo directo.

## 5. Evaluación de calidad

La versión inicial era funcional solo para entradas simples, pero tenía una estructura difícil de mantener. `tokenize` mezclaba recorrido, clasificación y presentación de depuración, mientras que `readWord` y `readNumber` dependían de atributos mutables. Las pruebas hicieron visibles defectos de comportamiento, no solo problemas estilísticos: una palabra que comienza por `let` era marcada como reservada, los identificadores con guion bajo se partían, los números perdían la coincidencia más larga, los errores desaparecían y una segunda llamada reutilizaba tokens de la primera.

La solución final es más testeable porque cada llamada trabaja con estado local, el catálogo se inyecta y la clasificación de palabras se puede aislar con Mockito. También es más extensible: `symbolType` centraliza los símbolos y los predicados ASCII documentan las reglas del minilenguaje. LCOM1 baja de 6 a 1 y CBO de 7 a 6, señales compatibles con una clase más enfocada y con menos dependencias. La CC máxima baja de 11 a 6, aunque la suma sube de 21 a 32 porque ahora las reglas ASCII y las condiciones de lectura son explícitas. Asimismo, LOC aumenta de 68 a 80; esto no contradice una mejora, pues parte del costo corresponde a validaciones que antes estaban implícitas en `Character.isLetter` y `Character.isDigit`.

Las métricas no capturan por sí solas la claridad de los nombres, la calidad de los mensajes de error, la cobertura de escenarios futuros ni la facilidad de integración. Por ejemplo, el `switch` tiene la CC máxima, pero es más sencillo de entender que una cadena de condicionales. La solución final es mantenible y testeable para el alcance definido: los 17 tests pasan y cubren casos normales, fronteras, errores y dependencia inyectada. Su extensibilidad sigue siendo deliberadamente

limitada a los tokens especificados; comentarios, decimales y operadores nuevos requerirían ampliar el contrato y las pruebas.

## 6. Cómo reproducir

Desde `analizador-lexico`:

```
mvn test
python3 scripts/metrics.py \
  --package src/main/java/cl/edu/metrics/lexer \
  --unit LexicalAnalyzer.java \
  --label Final
```

La ejecución final registra 17 pruebas aprobadas y `BUILD SUCCESS`.